Obsluga plikow w C#

System.IO | Formaty | Wyjatki | Cwiczenia

Kompletny przewodnik dla programistow .NET

Podstawy - przestrzen nazw System.IO

Przestrzen nazw **System.IO** zawiera klasy do odczytu, zapisu, tworzenia, usuwania i nawigowania po systemie plików.

Klasa / typ	Przeznaczenie
`File`	Statyczne metody szybkiego odczytu/zapisu (ReadAllText, WriteAllLines...)
`FileInfo`	Obiektowy dostęp do metadanych i operacji na pliku
`Directory`	Tworzenie, usuwanie, listowanie katalogów (statyczna)
`DirectoryInfo`	Obiektowy odpowiednik Directory
`Path`	Łączenie i parsowanie ścieżek (Combine, GetExtension...)
`StreamReader`	Odczyt tekstu linia po linii z dużych plików
`StreamWriter`	Zapis tekstu strumieniowo (oszczędny pamięciowo)
`FileStream`	Niskopoziomowy strumień bajtów - pliki binarne
`BinaryReader/Writer`	Odczyt/zapis typów pierwotnych (int, double...) binarnie
`MemoryStream`	Strumień w pamięci - buforowanie, testy

Zasada wyboru: Do krotkich plikow tekstowych uzywaj **File** (jednolinijkowy zapis/odczyt). Do duzych plikow lub binarnych zawsze **StreamReader/Writer** lub **FileStream**, aby nie ladowac calego pliku do pamieci.

Popularne formaty plikow w aplikacjach .NET

Format	Rozszerzenie	Biblioteka / klasa	Zastosowanie
`Tekst plain`	`.txt`	File, StreamReader	Logi, konfiguracje, eksport
`CSV`	`.csv`	File, CsvHelper (NuGet)	Dane tabelaryczne, import/eksport
`JSON`	`.json`	System.Text.Json, Newtonsoft.Json	API, konfiguracja, dane strukturalne
`XML`	`.xml`	XmlDocument, XDocument, XmlSerializer	Konfiguracje legacy, SOAP
`INI / YAML`	`.ini .yaml`	Microsoft.Extensions.Configuration	Konfiguracja aplikacji
`Binarny`	`dowolny`	FileStream, BinaryReader/Writer	Protokoły, dane gier, cache
`PDF`	`.pdf`	iTextSharp, PdfSharp (NuGet)	Raporty, dokumenty do druku
`Excel`	`.xlsx`	EPPlus, ClosedXML (NuGet)	Raporty, arkusze kalkulacyjne
`SQLite`	`.db .sqlite`	Microsoft.Data.Sqlite	Lokalna baza danych w pliku

NuGet do instalacji: `dotnet add package Newtonsoft.Json` | `dotnet add package CsvHelper` | `dotnet add package EPPlus`

Odczyt i zapis pliku tekstowego

Klasa File - szybkie operacje (małe pliki)

```
  // File - szybkie API
using System.IO;

// Odczyt calego pliku jako string
string content = File.ReadAllText("dane.txt");

// Odczyt jako tablica wierszy
string[] lines = File.ReadAllLines("dane.txt");
  foreach (string line in lines)
  Console.WriteLine(line);

// Zapis (nadpisuje plik)
  File.WriteAllText("wynik.txt", "Hello, C#!");

// Dopisywanie do istniejacego pliku
```

StreamReader / StreamWriter - duże pliki, linia po linii

```
  // StreamReader / StreamWriter
// Odczyt strumieniowy
using var reader = new StreamReader("duzy_plik.txt", Encoding.UTF8);
  string? line;
while ((line = reader.ReadLine()) != null)
  {
// przetwarzaj linie - nie ladujesz calosci do pamieci
  Console.WriteLine(line);
  }

// Zapis strumieniowy
using var writer = new StreamWriter("raport.txt", append: false, Encoding.UTF8);
  writer.WriteLine("Naglowek raportu");
for (int i = 1; i <= 1000; i++)
  writer.WriteLine($"Wiersz {i}: dane...");
// using automatycznie wywoluje Dispose() -> Flush() -> zamkniecie
```

> **Wydajnosc:** `File.ReadAllText` wczytuje caly plik do pamieci - idealne dla plikow <10 MB. Dla logow, CSV z milionami wierszy zawsze preferuj `StreamReader` z petla `ReadLine()`.

Obsluga wyjatkow przy pracy z plikami

Wyjatek	Kiedy wystapi
`FileNotFoundException`	Plik o podanej ścieżce nie istnieje
`DirectoryNotFoundException`	Katalog w ścieżce nie istnieje
`UnauthorizedAccessException`	Brak uprawnień do odczytu/zapisu
`IOException`	Błąd I/O: plik zablokowany, dysk pełny itp.
`PathTooLongException`	Ścieżka przekracza limit systemu plików
`ArgumentNullException`	Przekazano null jako ścieżkę
`NotSupportedException`	Nieobsługiwany format lub operacja
`OutOfMemoryException`	Za duzy plik załadowany do pamięci naraz

Wzorzec pełnej obslugi wyjątków

```
  // Bezpieczne wczytywanie pliku
static string? WczytajPlik(string sciezka)
  {
try
  {
if (!File.Exists(sciezka))
throw new FileNotFoundException($"Nie znaleziono: {sciezka}");

return File.ReadAllText(sciezka, Encoding.UTF8);
  }
catch (FileNotFoundException ex)
  {
Console.Error.WriteLine($"BLAD: {ex.Message}");
  return null;
  }
catch (UnauthorizedAccessException)
  {
Console.Error.WriteLine("BLAD: Brak uprawnien do pliku.");
  return null;
  }
catch (IOException ex)
  {
Console.Error.WriteLine($"BLAD I/O: {ex.Message}");
  return null;
  }
// nie lapaj Exception - niech inne bledy propaguja!
  }
```

Anty-wzorzec: catch (Exception ex) { /* ignoruj */ } - polykanie wyjatkow ukrywa bledy! Zawsze loguj wyjatek lub rzuc go dalej przez `throw;` (nie `throw ex;` - to kasuje stack trace).